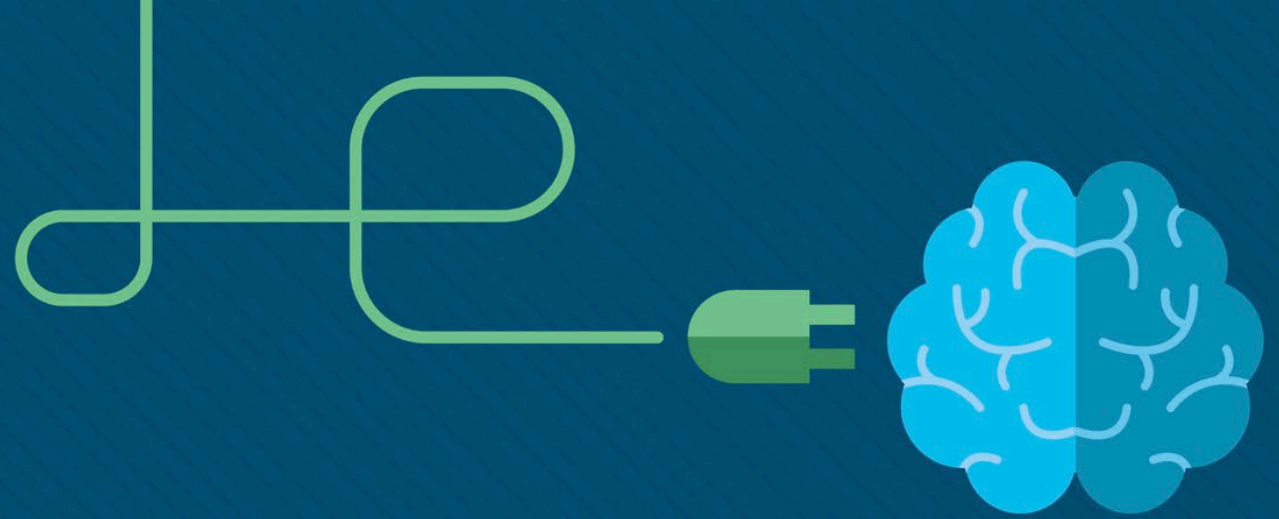




Week 1

RH124 Chapter 1: Getting Started on Red Hat Enterprise Linux

RH124 Chapter 2: Accessing the Command Line



1 The All-Important Shell

- Allows the user to communicate with the operating system without worrying about the underlying complexity of the communication.

For example, when you save a file to the storage disk with a simple shell command, you do not have to worry about which storage location to store the file at and how large files need to be split up to be stored over several discontinuous storage locations.

- Sometimes called an interpreter – takes user command, interprets it and relays the necessary command to the kernel.
- Two types of shell – Graphical User Interface (GUI) and Command Line Interface (CLI). Both Windows and Linux have both types of shell.

1.1 Bash (Bourne Again Shell)

- Default shell for Linux
- Interprets commands entered by the user in the CLI and translate into actions for the kernel to take
- Other shells available on a Linux operating system are the Bourne shell and C shell

See https://www.gnu.org/software/bash/manual/html_node/Major-Differences-From-The-Bourne-Shell.html for the key differences between Bash and Bourne shell

- Users interact with the Bash shell by executing commands which are interpreted by the shell. These actions may or may not return information to the command line depending on the command issued and its result.
- You can access the shell from the GUI by launching an application called a Terminal - 
- The general syntax of commands are as follows:

```
command [ options ] [ arguments ]
```

Note: many commands require options and arguments while others, like **ls** can be used alone.

1.2 What are commands?

- Option 1: a software programme that takes input from the user/system/files, manipulates some form of data and takes some action. The result of the action may/may not be visible on the screen. Action may be to manipulate files / system data which does not produce any visible output on the screen.
- Option 2: source of the command. A command can be from the following sources:
 - **Internal Commands**: Also called *built-in commands*, these commands are built-in to the shell itself. A good example is the `cd` (change directory) command as it is part of the Bash shell. When a user types the `cd` command, the Bash shell is already executing and knows how to interpret that command, requiring no additional programs to be started.
 - **External Commands**: These commands are stored in files that are searched by the shell. If you type the `ls` command, then the shell searches through a predetermined list of directories to try to find a file named `ls` that it can execute. These commands can also be executed by typing the complete path to the command.
 - **Aliases**: An alias can override a built-in command, function, or a command that is found in a file. Aliases can be useful for creating new commands built from existing functions and commands.
 - **Functions**: Functions can also be built using existing commands to either create new commands, override commands built-in to the shell or commands stored in files. Aliases and functions are normally loaded from the initialization files when the shell first starts, discussed later in this section.

1.3 Basic Command Syntax

- The general syntax of commands are as follows:

```
command [ options ] [ arguments ]
```

Options can be used with commands to expand or modify the way a command behaves. If it is necessary to add options, they can be specified after the command name. *Short options* are specified with a hyphen – followed by a single character. Short options are how options were traditionally specified.

An *argument* can be used to specify something for the command to act upon. Following a command, any desired arguments are allowed or are required depending on the command.

- Commands, options and arguments (in fact almost everything in Linux) is case sensitive. Hence typing *ls* is different from *LS*.

1.3.1 Specifying Options

- In the example below, executing the ls command without any options produces the result in the figure:

```
sysadmin@localhost:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

- However, executing the ls command with the `-l` short option produces the result with more information as shown in the figure:

```
sysadmin@localhost:~$ ls -l
total 32
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Desktop
drwxr-xr-x 4 sysadmin sysadmin 4096 Apr 24 2019 Documents
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Downloads
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Music
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Pictures
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Public
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Templates
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Videos
```

1.3.1 Specifying Options (Cont'd)

- Short options can also be combined. For example, running the `ls -l -r` (top figure) produces the same output as `ls -lr` (bottom figure)

```
sysadmin@localhost:~$ ls -l -r
total 32
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Videos
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Templates
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Public
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Pictures
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Music
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Downloads
drwxr-xr-x 4 sysadmin sysadmin 4096 Apr 24 2019 Documents
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Desktop
```

```
sysadmin@localhost:~$ ls -lr
total 32
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Videos
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Templates
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Public
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Pictures
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Music
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Downloads
drwxr-xr-x 4 sysadmin sysadmin 4096 Apr 24 2019 Documents
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Desktop
```

1.3.1 Specifying Options (Cont'd)

- Some options can also be specified in long format using a `--`. Example, `ls -r` when expressed in long option format is `ls --reverse`.

```
sysadmin@localhost:~$ ls --reverse
Videos  Templates  Public  Pictures  Music  Downloads  Documents  Desktop
sysadmin@localhost:~$ ls -r
Videos  Templates  Public  Pictures  Music  Downloads  Documents  Desktop
```

- Some options require arguments to follow. In such cases, it is not possible to combine the options.

```
sysadmin@localhost:~$ ls -w 40 -T 12
Desktop Music    Templates
Documents Pictures Videos
Downloads Public
```

Note: the `-w` option specifies the width of the output of the `ls` command while the `-T` option specifies the tab size. The above example forces the output to be 40 characters wide and the tab size to be 12 characters.

1.3.2 Specifying Arguments

- An example of specifying an argument is again using the *ls* command. The figure below shows an instance of running the *ls* command without any arguments. *ls* then take the current directory as a default argument and list its contents.

```
sysadmin@localhost:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

- The figure here shows the running of the *ls* command with an argument */etc/ppp*. In this case, the contents of the */etc/ppp* is listed instead of the current directory.

```
sysadmin@localhost:~$ ls /etc/ppp
ip-down.d  ip-up.d
```

1.3.2 Specifying Arguments (Cont'd)

- The ls command can also take more than one argument as follows:

```
sysadmin@localhost:~$ ls /etc/ppp /etc/ssh
/etc/ppp:
ip-down.d  ip-up.d

/etc/ssh:
moduli          ssh_host_ecdsa_key.pub  ssh_host_rsa_key      sshd_config
ssh_config      ssh_host_ed25519_key    ssh_host_rsa_key.pub
ssh_host_ecdsa_key  ssh_host_ed25519_key.pub  ssh_import_id
```

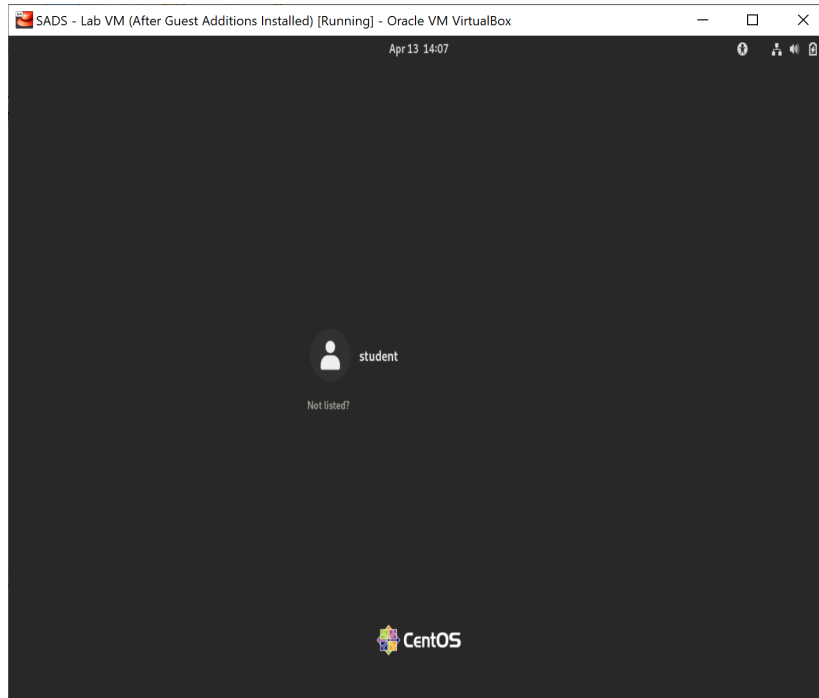
1.4 Log in to a Local System

A *terminal* is a text-based interface to enter commands into and print output from a computer system. To run the *shell*, you must log in to the computer on a terminal.

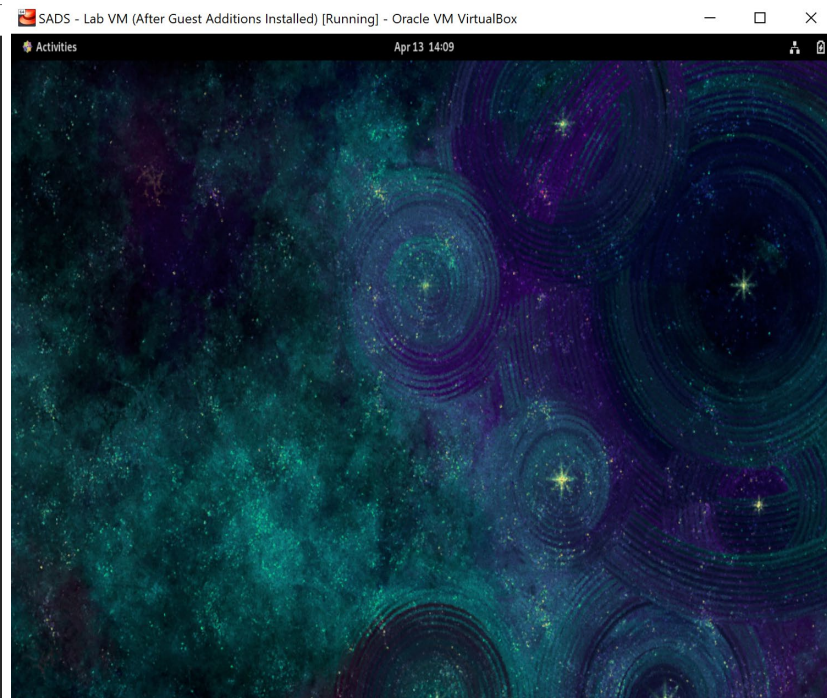
A hardware keyboard and display for input and output might be directly connected to the computer. This is the *physical console* from the Linux machine. The physical console supports multiple *virtual consoles*, which can run on separate terminals. Each virtual console supports an independent login session. You can switch between the virtual consoles by pressing Ctrl+Alt and a function key (F1 through F6) at the same time. Most of these virtual consoles run a terminal that provides a text login prompt. If you enter your username and password correctly, then you log in and get a shell prompt.

The computer might provide a graphical login prompt on one of the virtual consoles. You can use the graphical login prompt to log in to a graphical environment. The graphical environment also runs on a virtual console. To get a shell prompt, you must start a terminal program in the graphical environment. The shell prompt is provided in an application window of your graphical terminal program.

1.4 Log in to a Local System (cont'd)

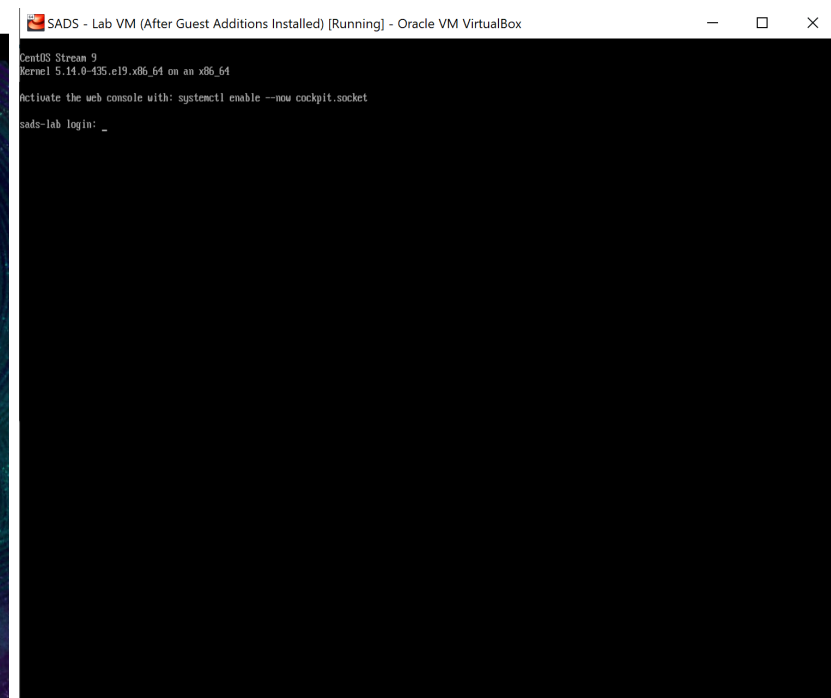


Ctrl-Alt-F1



Ctrl-Alt-F2

[if you login to the GUI first before launching
any of the text-based virtual consoles]



Ctrl-Alt-F3 to F6

2. Executing commands on the Command Line

Basic Command Syntax

- The GNU Bourne-Again Shell (bash) is a program that interprets commands that the user types. Each string that is typed into the shell can have up to three parts: the command, options (which usually begin with a hyphen - or double hyphen -- characters), and arguments. Each word that is typed into the shell is separated from other words with spaces. Commands are the names of programs that are installed on the system. Each command has its options and arguments.
- When you are ready to execute a command, press the Enter key. Type each command on a separate line. The command output is displayed before the following shell prompt appears.
- To type more than one command on a single line, use the semicolon (;) as a command separator. A semicolon is a member of a class of characters called metacharacters that have a special interpretation for bash.

```
[root@sads-lab ~]# whoami  
root  
[root@sads-lab ~]# █
```

```
[root@sads-lab ~]# whoami; ls  
root  
anaconda-ks.cfg  Documents  Music      Public     Videos  
Desktop          Downloads  Pictures   Templates  
[root@sads-lab ~]# █
```

2. Executing commands on the Command Line (cont'd)

- The **date** command displays the current date and time. The superuser or a privileged user can also use the date command to set the system clock. Use the plus sign (+) as an argument to specify a format string for the date command.

```
[root@sads-lab ~]# date
Sat 13 Apr 2024 02:29:36 PM +08
[root@sads-lab ~]# date +%R
14:29
[root@sads-lab ~]# date +%x
13/04/2024
[root@sads-lab ~]#
```

- The **passwd** command is used to update a user's password. Users can only change their own passwords while the *root* user can update the password for any user. In the example to the right, the *sysadmin* account is prevented from attempting to update *dev1*'s account while both the *root* user or the *dev1* user can successfully update the password

```
sysadmin@localhost:~$ passwd dev1
passwd: You may not view or modify password information for dev1.
sysadmin@localhost:~$ su -
Password:
root@localhost:~# passwd dev1
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@localhost:~# su - dev1
dev1@localhost:~$ passwd
Changing password for dev1.
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

2. Executing commands on the Command Line (cont'd)

- Linux does not require file name extensions to classify files by type. The **file** command scans the compiled header of a file for a 2-digit magic number and displays its type. Text files are recognized because they are not compiled.

```
[root@sads-lab ~]# file /etc/passwd
/etc/passwd: ASCII text
[root@sads-lab ~]# file /bin/passwd
/bin/passwd: setuid ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=a467cb9c8fa7306d41b96a820b
0178f3a9c66055, for GNU/Linux 3.2.0, stripped
[root@sads-lab ~]#
```

2. Executing commands on the Command Line (cont'd)

- With a single file argument, the **cat** command will simply output the contents of a file

```
cat [OPTION]... [FILE]...
```

```
sysadmin@localhost:~$ cd Documents/  
sysadmin@localhost:~/Documents$ cat alpha-first.txt  
A is for Animal  
B is for Bear  
C is for Cat  
D is for Dog  
E is for Elephant  
F is for Flower
```

```
sysadmin@localhost:~/Documents$ cat alpha-second.txt  
G is for Grapes  
H is for Happy  
I is for Ink  
J is for Juice  
K is for Kangaroo  
L is for Lol  
M is for Monkey
```

- However, with two or more file names as arguments to the **cat** command, the file contents are concatenated and displayed on the screen (next slide).

2. Executing commands on the Command Line (cont'd)

```
sysadmin@localhost:~/Documents$ cat alpha-first.txt alpha-second.txt
```

```
A is for Animal
```

```
B is for Bear
```

```
C is for Cat
```

```
D is for Dog
```

```
E is for Elephant
```

```
F is for Flower
```

```
G is for Grapes
```

```
H is for Happy
```

```
I is for Ink
```

```
J is for Juice
```

```
K is for Kangaroo
```

```
L is for Lol
```

```
M is for Monkey
```

2. Executing commands on the Command Line (cont'd)

- The **cat** command will display the full contents of files on the screen without consideration for length of output. For the normal user, it can be frustrating as file contents that extend beyond one screen page will scroll off the screen.
- Bash provides the **less** and **more** commands to allow more controlled viewing of file contents. They allow users to navigate the document using keystroke commands. For example, when using **less** as a pager, to advance forward a page, press the `Spacebar`. Use the `H` key or `Shift+H` to display a help screen with a summary of the **less** commands.

```
less FILE
```

- To exit the **less** command, use the `Q` key.

2. Executing commands on the Command Line (cont'd)

- The purpose of the **head** command is to view the beginning of a file or output

```
head [OPTION]... [FILE]...
```

- By default, the **head** command will display the first ten lines of a file's contents. You can use a number as an option to indicate how many lines output to display. For example, the figure below show how to use **head** to display the first 5 lines of the alpha.txt file.

```
sysadmin@localhost:~/Documents$ head alpha.txt
A is for Apple
B is for Bear
C is for Cat
D is for Dog
E is for Elephant
F is for Flower
G is for Grapes
H is for Happy
I is for Ink
J is for Juice
```

```
sysadmin@localhost:~/Documents$ head -5 alpha.txt
A is for Apple
B is for Bear
C is for Cat
D is for Dog
E is for Elephant
```

2. Executing commands on the Command Line (cont'd)

- Finally, you can use the *-n* option with a negative number to omit the n number of lines from the top of the file. The example will print out all and omit the last 10 lines from the *alpha.txt* file.

```
sysadmin@localhost:~/Documents$ head -n -10 alpha.txt
A is for Apple
B is for Bear
C is for Cat
D is for Dog
E is for Elephant
F is for Flower
G is for Grapes
H is for Happy
I is for Ink
J is for Juice
K is for Kangaroo
L is for Lol
M is for Monkey
N is for Nickel
O is for Oval
P is for Pickle
```

2. Executing commands on the Command Line (cont'd)

- The opposite of the **head** command, the **tail** command displays contents from the end of the file and not the beginning.

```
head [OPTION]... [FILE]...
```

- By default, the **tail** command displays ten lines when no options are given (refer to figure on top right).
- You may use either a number or the **-n** option as an argument to the **tail** command to specify how many lines you want to output from the end of the file. Therefore, the following two commands are equivalent (middle and bottom right).

```
sysadmin@localhost:~/Documents$ tail alpha.txt
Q is for Quark
R is for Rat
S is for Sloth
T is for Turnip
U is for Up
V is for Velvet
W is for Walrus
X is for Xenon
Y is for Yellow
Z is for Zebra
```

```
sysadmin@localhost:~/Documents$ tail -3 alpha.txt
X is for Xenon
Y is for Yellow
Z is for Zebra
sysadmin@localhost:~/Documents$ tail -n3 alpha.txt
X is for Xenon
Y is for Yellow
Z is for Zebra
```

2. Executing commands on the Command Line (cont'd)

- When using the *-n* option with a number prefixed by a plus sign + character, the number is interpreted as the line number in the file to start displaying content, ie, it will display from that line to the end of the file. In other words, the *-n* option used with the +20 argument will display the contents of the file starting at line 20 and continuing until the end of the file.

```
sysadmin@localhost:~/Documents$ tail -n +20 alpha.txt
T is for Turnip
U is for Up
V is for Velvet
W is for Walrus
X is for Xenon
Y is for Yellow
Z is for Zebra
```

2. Executing commands on the Command Line (cont'd)

- The **wc** command can be used to analyze a text file. By default, **wc** counts the number of lines, words and bytes in a passage and outputs this information to the screen

```
sysadmin@localhost:~$ cat Documents/alpha-first.txt
A is for Apple
B is for Bear
C is for Cat
D is for Dog
E is for Elephant
F is for Flower

sysadmin@localhost:~$ wc Documents/alpha-first.txt
6 24 89 Documents/alpha-first.txt
```

- The three numbers reported by **wc** is 6 lines, 24 words and 90 bytes in length. To number of bytes include spaces, punctuation, and carriage returns.
- Use the **wc** command with the **-l** option to only see the number of lines, **-w** to count the number of words and **-m** to count the number of bytes.
- Finally **-L** option returns the length of the longest line in the file.

2. Executing commands on the Command Line (cont'd)

- Commands with many options and arguments can quickly become long and are automatically wrapped by the command window when the cursor reaches the right margin. Instead, type a long command by using more than one line for easier reading.
- To write one command in more than one line, use a backslash character (\), which is referred to as the escape character. The backslash character ignores the meaning of the following character. This way, the shell acknowledges the request by displaying a continuation prompt on an empty new line, which is known as the secondary prompt, and uses the greater-than character (>) by default.

```
[root@sads-lab ~]# head -n 3 \  
> /usr/share/dict/words \  
> /usr/share/dict/linux.words  
==> /usr/share/dict/words <==  
1080  
10-point  
10th  
  
==> /usr/share/dict/linux.words <==  
1080  
10-point  
10th  
[root@sads-lab ~]# █
```

3 Command Completion

- A useful tool of the Bash shell is the ability to complete commands and their arguments automatically. By that it means that by typing a few characters of a command or argument and then pressing Tab key, Bash will automatically complete the command or file name if the sequence of typed characters can uniquely identify the command or file name.
- If there are more than one match to the sequence of characters typed, then pressing the Tab key a second time list the possible matches so far.

```
sysadmin@localhost:~$ ca
cal                capsh                cat                cautious-launcher
calendar           captainfo           catchsegv
caller            case                catman
sysadmin@localhost:~$ ca
```

- The Tab key is a useful productivity feature that helps avoid typing errors when typing long command and / or file names.

4 Command History

- The `~/.bash_history` contains a history of the commands that a user has executed within the Bash shell. When a user exits the Bash shell, it writes out the recent history to this file.
- The Bash shell allows a user to use the Up ↑ and Down ↓ keys to cycle through the list of previous commands executed at the command line. Once a previous command is found, the user can press the Enter key to execute it.
- The user can also use the Ctrl+R key combination to perform a search of the history list.

4.1 Using the History Command

- The **history** command can be used to re-execute previously executed commands. When used with no arguments, the **history** command provides a list of previously executed commands.
- Run the **history** command with the **-c** option to clear the history list.
- Run the **history** command with a number as an argument to limit the number of history entries to display.

```
sysadmin@localhost:~$ history 5
```

```
12 cat ~/.bash_logout
13 cat /etc/bash_logout
14 clear
15 history
16 history 5
```

```
sysadmin@localhost:~$ history
```

```
1 clear
2 echo $PATH
3 PATH=$PATH:/etc/bin
4 echo $PATH
5 clear
6 PATH=/etc/bin
7 exit
8 echo $PATH
9 cat ~/.bash_logout
10 cat /etc/bash_logout
11 clear
12 cat ~/.bash_logout
13 cat /etc/bash_logout
14 clear
15 history
```

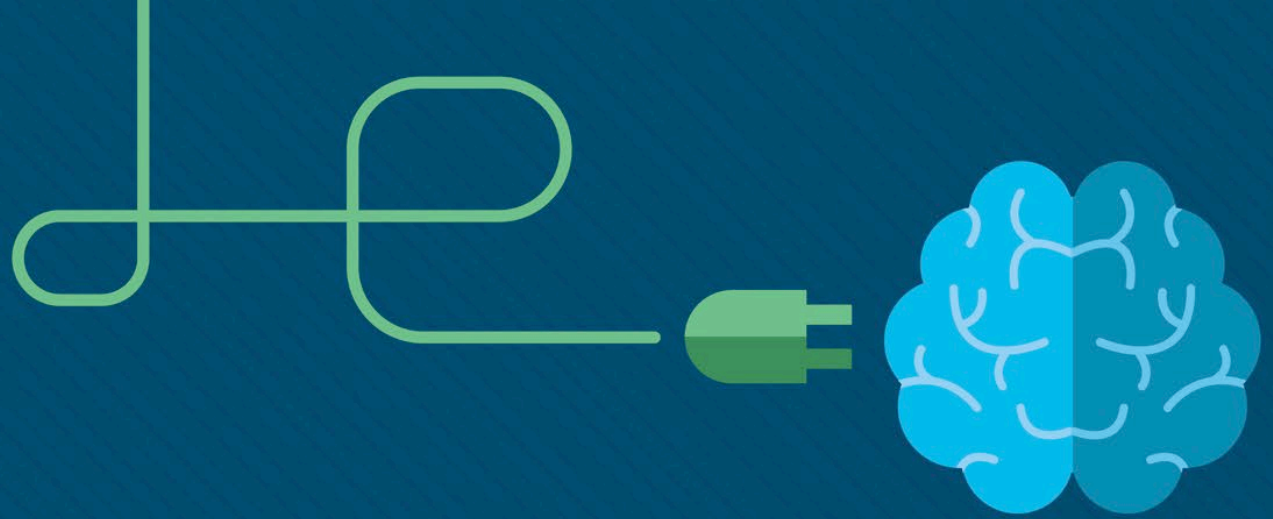
```
sysadmin@localhost:~$
```

4.2 Executing Previous Commands

```
sysadmin@localhost:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
sysadmin@localhost:~$ !!
ls
Desktop Documents Downloads Music Pictures Public Templates Videos
```

- The **!** exclamation mark is a special character to the Bash shell to indicate the execution of a command within the history list. There are many ways to use the **!** exclamation character to re-execute commands; for example, in the figure above, executing two exclamation characters will repeat the previous command

History Command	Meaning
!!	Repeat the last command
!-4	Execute the command that was run four commands ago
!555	Execute command number 555
!ec	Execute the last command that started with ec



Lab

RH124 Chapter 2.7 Lab: Access the Command Line



Practice Exercises

1 Accessing the Shell & Command Line

- Open a terminal and identify the default shell.
- Switch to a different virtual console using Ctrl+Alt+F3. How do you return to GUI mode?

2 Basic Command Execution & Syntax

- Run a command to display today's date and time.
- Use ls to list files in your current directory. Add the -l and -r options—what changes?
- Execute two commands in a single line using a separator.

3 Command History & Shortcuts

- View the last 10 executed commands.
- Re-run the most recent command without typing it again.
- Use Ctrl+R to search for a previously executed command.

4 Understanding Options & Arguments

- Run ls with an argument to list files inside /etc/.
- Use a long-format option (--reverse) instead of a short option (-r). What happens?
- Try ls -w 40 -T 12. What do these options do?